

Analizor de trafic HTTP

Documentatie tehnica - proiect la Retele de Calculatoare

1. Ce este si ce face

Am facut un proxy de securitate care sta in fata unei aplicatii web si se uita la fiecare cerere care intra. Pentru fiecare cerere incearca sa-si dea seama cat de riscanta pare, o pune intr-o categorie (scraper, scanare, injectie, brute force, traversare de directoare, comportament de tip DoS), explica de ce a marcat-o si abia apoi o trimite mai departe catre aplicatia adevarata. Totul ruleaza local.

Practic, in loc sa expun aplicatia direct, am pus in fata un paznic care vede tot traficul si il analizeaza inainte.

2. Cum se ruleaza

De ce ai nevoie

- Python 3.10 sau mai nou
- optional: Ollama, daca vrei si partea cu model de limbaj

Instalare

```
python -m venv venv
venv\Scripts\activate      (pe Windows)
# source venv/bin/activate  (pe Linux / macOS)
pip install -r requirements.txt
```

Pornire (doua terminale)

In primul terminal pornesc aplicatia pe care o protejam (un mic magazin de test), pe portul 8001:

```
python aplicatie_web/magazin.py
```

In al doilea terminal pornesc proxy-ul, pe portul 8000:

```
python analizor/proxy.py
```

Dupa asta intru pe aplicatie PRIN proxy, din browser: <http://localhost:8000>

Panoul de monitorizare e la: <http://localhost:8000/dashboard>

Trafic de test

Ca sa vad detectia fara sa dau eu cereri de mana, pornesc scriptul care trimite singur trafic normal si cateva atacuri:

```
python teste/simulare_trafic.py --scenario all
```

Optional: Ollama

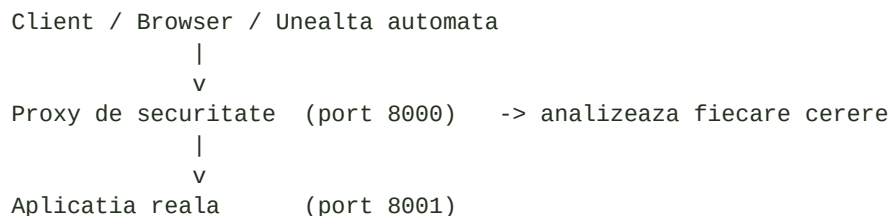
Daca vreau si modelul de limbaj, pornesc Ollama si descarc un model. Daca am deja alt model, il pot folosi printr-o variabila de mediu, fara sa modific codul. Daca Ollama nu ruleaza, sistemul merge doar pe reguli +

anomalii.

```
ollama serve
ollama pull llama3.2
set OLLAMA_MODEL=llama3.1:8b      (daca am deja acest model)
```

Modelul de anomalii se antreneaza singur la prima pornire. Il pot antrena si separat: python analizor/model_anomalii.py

3. Arhitectura



Proxy-ul e facut in FastAPI, cu o ruta care prinde tot traficul, si transmite mai departe cererile cu httpx.

4. Cum functioneaza (pas cu pas)

- Proxy-ul primeste cererea pe 8000 si scoate datele din ea (IP, metoda, cale, query, user-agent, dimensiune corp).
- Se calculeaza caracteristicile de comportament ale IP-ului, pe o fereastră de 60 de secunde.
- Regulile verifica tipare cunoscute si dau un scor + categorii.
- Modelul de anomalii (Isolation Forest) da un scor de anomalie.
- Daca scorul preliminar e deja ridicat si Ollama e pornit, modelul de limbaj da o clasificare in plus.
- Se calculeaza scorul final si verdictul.
- Daca IP-ul e blocat, cererea primeste 403 si nu mai e transmisa. Altfel, ajunge la aplicatia reala.
- Cererea, scorul si explicatia se salveaza; daca scorul e foarte mare, IP-ul e blocat temporar.
- Raspunsul aplicatiei ajunge inapoi la client, iar panoul afiseaza totul aproape in timp real.

5. Analiza: reguli plus doua modele AI

Analiza nu se bazeaza pe un singur lucru, ci pe trei surse care lucreaza impreuna, iar scorurile lor se combina. Doua dintre ele sunt modele de inteligenta artificiala, a treia e un set de reguli scrise de mine. Asa, decizia nu vine de la o singura componenta.

a) Regulile (nu sunt AI)

Verificari rapide si clare: cai sensibile (/admin, /.env, /wp-admin), tipare de injectie SQL / XSS / comenzi, traversare de directoare, user-agent de unelte automate, rata mare de cereri, multe 404, autentificari esuate.

b) Modelul 1 (AI clasic): Isolation Forest

Un model din scikit-learn, antrenat pe trafic normal generat sintetic. Invata cum arata comportamentul obisnuit si marcheaza ce iese mult din tipar. Da un scor de anomalie de la 0 la 100 si un motiv.

c) Modelul 2 (model de limbaj): LLM local prin Ollama

Un model de limbaj rulat local prin Ollama. Primeste detaliile cererii si raspunde in format JSON, cu: is_suspicious, category, risk_score, reason, possible_intent si recommended_action. Promptul e defensiv (doar clasifica si explica). Il chem doar cand scorul preliminar e deja ridicat, ca sa nu incetinesc traficul normal. Daca Ollama nu e pornit, partea asta se sare automat.

Ca sa fie clar: sunt doua modele AI plus reguli, nu un grup de agenti separati. Ambele modele chiar ruleaza si contribuie la scorul final - le-am testat, nu sunt simulate.

6. Caracteristicile de comportament

Pentru fiecare IP, pe fereastra de 60 de secunde, se calculeaza 10 valori numerice care intra in modelul de anomalii:

Caracteristica	Ce inseamna
requests_per_minute	cate cereri a facut IP-ul in ultimul minut
unique_paths_per_minute	cate cai diferite a accesat
percentage_404	procentul de raspunsuri 404
average_response_size	dimensiunea medie a raspunsurilor
average_latency_ms	latenta medie
user_agent_length	lungimea user-agent-ului
number_of_query_parameters	cati parametri are in query
path_depth	adancimea caii
body_size	dimensiunea corpului cererii
failed_login_count	cate autentificari esuate

7. Categorii de detectie

Categorie	Cum o detectez
scraper	rata mare, multe cai unice, user-agent automat
vulnerability_probe / scanner	cai sensibile si multe raspunsuri 404
sensitive_file_probe	cereri catre fisiere de configurare (.env, .git, .sql)
injection_attempt	tipare de SQL injection, command injection sau XSS
path_traversal	secvente de tip ../ catre fisiere de sistem
brute_force	multe autentificari esuate
dos_like_behavior	rata foarte mare de cereri sau payload mare
unknown_anomaly	ceva marcat de modelul de anomalii, fara o regula clara

8. Scorul de risc

Scorul final combina cele trei surse:

- cu LLM: final = $0.40 * \text{reguli} + 0.25 * \text{anomalie} + 0.35 * \text{llm}$
- fara LLM: final = $0.60 * \text{reguli} + 0.40 * \text{anomalie}$
- daca o regula clara a dat scor mare (peste 80), scorul final nu coboara sub 80

Verdictul vine din scorul final:

Scor	Verdict
0 - 30	normal
31 - 60	suspect
61 - 80	risc ridicat
81 - 100	critic

La un scor foarte mare (peste 85) IP-ul e blocat temporar (10 minute).

9. Ce date se salveaza

Fiecare cerere analizata se salveaza intr-o baza de date SQLite, cu: ora, IP, metoda, cale, query, user-agent, dimensiunea corpului, codul de raspuns, latentă, dimensiunea raspunsului, scorurile (reguli, anomalie, LLM, final), verdictul, categoria, motivul, intentia posibila si actiunea recomandata. In plus, fiecare cerere se scrie si intr-un fisier de log.

10. Panou si rapoarte

Panoul de monitorizare e servit chiar de proxy la /dashboard. Arata statistici generale, distributia pe categorii, IP-urile cu scor relevant si cererile recente (cu scor, verdict, categorie si motiv), plus un filtru pe scor minim. Se reimprospateaza singur la cateva secunde. Separat, pot genera rapoarte cu cererile flagate in trei formate: JSON, HTML si CSV.

11. Ce am folosit

Componenta	Tehnologie
Limbaj	Python 3.10+
Proxy / API / panou	FastAPI + Uvicorn
Client HTTP catre aplicatie	httpx (asincron)
Baza de date	SQLite (sqlite3)
Detectie de anomalii	scikit-learn (IsolationForest) + numpy + joblib
Model de limbaj (optional)	Ollama (apelat cu requests)
Simulator de trafic	requests

12. Cum e organizat proiectul

Folder / fisier	Ce contine
analizor/	proxy-ul si toata analiza (vezi mai jos)
aplicatie_web/magazin.py	aplicatia reala de test, pe portul 8001
teste/simulare_trafic.py	scriptul care trimite trafic de test
Documentatie.pdf	documentul acesta
requirements.txt	bibliotecile necesare

In folderul analizor/:

Fisier	RoI
proxy.py	proxy-ul si decizia finala
reguli.py	regulile de detectie
model_anomalii.py	modelul de anomalii (Isolation Forest)
model_llm.py	clasificatorul prin Ollama (optional)
caracteristici.py	caracteristicile de comportament per IP
baza_date.py	accesul la baza de date
jurnal.py	salvarea inregistrarilor
panou.py	panoul din browser
rapoarte.py	generarea rapoartelor
config.py	setarile (porturi, praguri, liste)

13. Limitari

Sistemul estimeaza riscul pe baza tiparelor de comportament, nu dovedeste ca o cerere e malitioasa. Unii boti sunt legitimi, iar unii atacatori pot parea utilizatori normali. Modelul de anomalii e antrenat pe trafic sintetic, deci pe trafic real ar trebui reantrenat. Caracteristicile sunt tinute in memorie, asa ca la repornire ferestrele se reseteaza. Pe scurt, rezultatul e un semnal de securitate, nu o dovada.